```
"""
eva/core.py — EVA Research Node core
orchestration
STATUS: RECONSTRUCTED, NOT CONFIRMED.

⚠️ This file does not exist in any uploaded
source. It was written to satisfy
the ✅-confirmed behavior in
EVA_ENGINEERING_SPEC.md / API_SPEC.md /
DATABASE_SCHEMA.md
(observed from api.py) and fills every ❓
open question with a flagged, reasonable
default. If the real core.py is ever
provided, THAT file wins per the One Rule —
this one gets discarded, not merged.

Confirmed behaviors this file must satisfy
(from api.py):
  - EVACore(db_path)
constructor
[CONFIRMED shape]
  - core.sentinel.review_task(task,
user_confirmed) -> dict    [CONFIRMED]
  - core.ghost.ingest(mission_id,
statement, source_id) -> record + conflicts
[CONFIRMED]
  - core.ghost.search(query, mission_id) ->
list               [CONFIRMED]
  - core.tools.list_tools() ->
list                           [CONFIRMED]
```

```
  - core.audit.history(mission_id=None) ->
list of events      [CONFIRMED]
  - core.get_mission_status(mission_id) ->
dict                [CONFIRMED to exist,
shape ASSUMED]
  - submit_mission(...) -> mission
dict                        [CONFIRMED via
/mission/create]

Open questions resolved here as ASSUMPTIONS
(see schema.sql for matching defaults):
  - DB engine:
SQLite
     [ASSUMED]
  - Pixie: implemented as a plain function,
not a class/module  [ASSUMED — Q5/Q6 from
dev spec]
  - Agent identity: enum only, no `agents`
table                [ASSUMED]
  - Gateway enforcement: Sentinel review is
mandatory for every tool call, fail-closed
[ASSUMED]
"""

import sqlite3
import uuid
from dataclasses import dataclass, field
from datetime import datetime, timezone
from enum import Enum
from typing import Any, Optional


def new_id() -> str:
    return uuid.uuid4().hex
```

```
def now_iso() -> str:
    return
datetime.now(timezone.utc).isoformat()


# ----------------------------------------
---------------------------------
# Enums — members are ASSUMED where Stage 1
docs marked them ❓ unconfirmed.
# ----------------------------------------
---------------------------------

class AgentName(str, Enum):
    PIXIE = "pixie"
    NOVA = "nova"
    SENTINEL = "sentinel"
    GHOST = "ghost"


class AuthLevel(str, Enum):
    USER_APPROVED = "USER_APPROVED"      #
[CONFIRMED — only known member]
    AUTO_APPROVED = "AUTO_APPROVED"      #
[ASSUMED]
    DENIED = "DENIED"                    #
[ASSUMED]


class TaskStatus(str, Enum):
    PENDING = "pending"                  #
[ASSUMED]
    AUTHORIZED = "authorized"            #
```

```
[ASSUMED]
    REFUSED = "refused"                 #
[ASSUMED]
    COMPLETE = "complete"               #
[ASSUMED]
    FAILED = "failed"                   #
[ASSUMED]


class MissionStatus(str, Enum):
    PENDING = "pending"                 #
[ASSUMED]
    IN_PROGRESS = "in_progress"         #
[ASSUMED]
    COMPLETE = "complete"               #
[ASSUMED]
    FAILED = "failed"                   #
[ASSUMED]


class Classification(str, Enum):
    VERIFIED = "verified"               #
[ASSUMED]
    UNVERIFIED = "unverified"           #
[ASSUMED]
    CONFLICTED = "conflicted"           #
[ASSUMED]


class Decision(str, Enum):
    ALLOW = "allow"
    DENY = "deny"
```

```
# ------------------------------------
---------------------------------
# Data shapes
# ------------------------------------
---------------------------------

@dataclass
class Task:
    id: str
    mission_id: str
    agent: str
    description: str
    auth_level: str =
AuthLevel.USER_APPROVED.value
    status: str = TaskStatus.PENDING.value


# ------------------------------------
---------------------------------
# Persistence boundary — the only module
that opens a DB connection.
# ------------------------------------
---------------------------------

class Database:
    def __init__(self, db_path: str =
"eva.db"):
        self.db_path = db_path
        self.conn =
sqlite3.connect(db_path,
check_same_thread=False)
        self.conn.row_factory = sqlite3.Row
        self._init_schema()
```

```
    def _init_schema(self):
        # Mirrors schema.sql. Kept inline
so core.py is self-contained;
        # schema.sql remains the source of
truth for migrations.
        self.conn.executescript("""
        CREATE TABLE IF NOT EXISTS users (
            id TEXT PRIMARY KEY, name TEXT
NOT NULL DEFAULT 'anonymous',
            created_at TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS missions
(
            id TEXT PRIMARY KEY, user_id
TEXT NOT NULL, objective TEXT NOT NULL,
            status TEXT NOT NULL DEFAULT
'pending', source_text TEXT,
            user_confirmed INTEGER NOT NULL
DEFAULT 0, report TEXT,
            created_at TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS
mission_sources (
            id TEXT PRIMARY KEY, mission_id
TEXT NOT NULL,
            origin TEXT, description TEXT,
text TEXT
        );
        CREATE TABLE IF NOT EXISTS tasks (
            id TEXT PRIMARY KEY, mission_id
TEXT NOT NULL, agent TEXT NOT NULL,
            description TEXT NOT NULL,
auth_level TEXT NOT NULL,
            status TEXT NOT NULL,
```

```
created_at TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS
memory_records (
            id TEXT PRIMARY KEY, mission_id
TEXT NOT NULL, content TEXT NOT NULL,
            classification TEXT NOT NULL,
confidence REAL NOT NULL,
            source_id TEXT, timestamp TEXT
NOT NULL
        );
        CREATE TABLE IF NOT EXISTS
memory_conflicts (
            id TEXT PRIMARY KEY,
memory_record_id TEXT NOT NULL,
            conflicting_record_id TEXT NOT
NULL
        );
        CREATE TABLE IF NOT EXISTS
audit_logs (
            id TEXT PRIMARY KEY, mission_id
TEXT, timestamp TEXT NOT NULL,
            agent TEXT NOT NULL, action
TEXT NOT NULL, result TEXT NOT NULL
        );
        """)
        self.conn.commit()


# ----------------------------------------
---------------------------------
# Sentinel — authorization / policy
adjudication.
# [ASSUMED] fail-closed: any exception or
```

```
unknown input -> DENY, never pass-through.
# ---------------------------------------
---------------------------------

class Sentinel:
    def __init__(self, db: Database):
        self.db = db

    def review_task(self, task: Task,
user_confirmed: bool = False) -> dict:
        """[CONFIRMED signature] ->
{decision, auth_level, reason}"""
        try:
            if task.agent not in (a.value
for a in AgentName):
                return self._deny(task,
"unknown agent")

            # [ASSUMED policy] sensitive
agents require explicit user confirmation
            if task.agent in
(AgentName.NOVA.value,) and not
user_confirmed:
                return self._deny(task,
"external research requires user
confirmation")

            self._audit(task.mission_id,
task.agent, "authorize", "allow")
            return {
                "decision":
Decision.ALLOW.value,
                "auth_level":
task.auth_level or
```

```
AuthLevel.USER_APPROVED.value,
                "reason": "policy check
passed",
            }
        except Exception as e:  # fail-
closed, per SB-1-style discipline
            self._audit(task.mission_id,
task.agent, "authorize", "deny")
            return {"decision":
Decision.DENY.value, "auth_level":
AuthLevel.DENIED.value,
                    "reason": f"error
during review: {e}"}

    def _deny(self, task: Task, reason:
str) -> dict:
        self._audit(task.mission_id,
task.agent, "authorize", "deny")
        return {"decision":
Decision.DENY.value, "auth_level":
AuthLevel.DENIED.value, "reason": reason}

    def _audit(self, mission_id: str,
agent: str, action: str, result: str):
        self.db.conn.execute(
            "INSERT INTO audit_logs (id,
mission_id, timestamp, agent, action,
result) VALUES (?,?,?,?,?,?)",
            (new_id(), mission_id,
now_iso(), agent, action, result),
        )
        self.db.conn.commit()
```

```
# ----------------------------------------
---------------------------------
# Ghost — validated memory storage.
# ----------------------------------------
---------------------------------

class Ghost:
    def __init__(self, db: Database):
        self.db = db

    def ingest(self, mission_id: str,
statement: str, source_id: Optional[str] =
None) -> dict:
        """[CONFIRMED signature] -> record
+ conflict list.
        [ASSUMED] `statement` input is
stored verbatim as `content`."""
        record_id = new_id()
        # [ASSUMED] naive
classification/confidence — real logic
unknown
        classification =
Classification.UNVERIFIED.value
        confidence = 0.5

        conflicts =
self._find_conflicts(mission_id, statement)

        self.db.conn.execute(
            "INSERT INTO memory_records
(id, mission_id, content, classification,
confidence, source_id, timestamp) "
            "VALUES (?,?,?,?,?,?,?)",
            (record_id, mission_id,
```

```
statement, classification, confidence,
source_id, now_iso()),
        )
        for c_id in conflicts:
            self.db.conn.execute(
                "INSERT INTO
memory_conflicts (id, memory_record_id,
conflicting_record_id) VALUES (?,?,?)",
                (new_id(), record_id,
c_id),
            )
        self.db.conn.commit()

        return {
            "id": record_id,
            "classification":
classification,
            "confidence": confidence,
            "conflicts": conflicts,
        }

    def _find_conflicts(self, mission_id:
str, statement: str) -> list:
        # [ASSUMED] placeholder — no real
conflict detection logic without core.py
        return []

    def search(self, query: str,
mission_id: Optional[str] = None) -> list:
        """[CONFIRMED signature] ->
{results: [{id, content, classification,
confidence}]}"""
        sql = "SELECT id, content,
classification, confidence FROM
```

```
memory_records WHERE content LIKE ?"
        params: list = [f"%{query}%"]
        if mission_id:
            sql += " AND mission_id = ?"
            params.append(mission_id)
        rows = self.db.conn.execute(sql,
params).fetchall()
        return [dict(r) for r in rows]


# ----------------------------------------
---------------------------------
# Tool Gateway — [ASSUMED] Sentinel review
is mandatory for every call, fail-closed.
# Real gateway.py enforcement logic is
unconfirmed; this is a minimal stand-in.
# ----------------------------------------
---------------------------------

class ToolGateway:
    def __init__(self, db: Database,
sentinel: Sentinel):
        self.db = db
        self.sentinel = sentinel
        self._tools = {
            "web_search": {"name":
"web_search", "description": "DuckDuckGo
search, no API key"},  # [CONFIRMED to
exist]
        }

    def list_tools(self) -> list:
        return list(self._tools.values())
```

```
    def execute(self, agent: str,
tool_name: str, mission_id: str, params:
dict) -> dict:
        """[ASSUMED] mirrors
/tool/execute's frontend-observed
request/response shape."""
        if tool_name not in self._tools:
            return {"success": False,
"data": {"error": f"unknown tool:
{tool_name}"}}

        task = Task(id=new_id(),
mission_id=mission_id, agent=agent,
description=f"execute {tool_name}")
        decision =
self.sentinel.review_task(task,
user_confirmed=True)
        if decision["decision"] !=
Decision.ALLOW.value:
            return {"success": False,
"data": {"error": decision["reason"]}}

        # [ASSUMED] actual tool dispatch
stubbed — real implementation unknown
        return {"success": True, "data":
{"tool": tool_name, "params": params,
"note": "stub execution"}}


# ----------------------------------------
---------------------------------
# Audit — read access over audit_logs.
# ----------------------------------------
---------------------------------
```

```
class Audit:
    def __init__(self, db: Database):
        self.db = db

    def history(self, mission_id:
Optional[str] = None) -> list:
        """[CONFIRMED signature] -> events:
[{timestamp, agent, action, result}]"""
        sql = "SELECT timestamp, agent,
action, result FROM audit_logs"
        params: list = []
        if mission_id:
            sql += " WHERE mission_id = ?"
            params.append(mission_id)
        sql += " ORDER BY timestamp ASC"
        rows = self.db.conn.execute(sql,
params).fetchall()
        return [dict(r) for r in rows]


# ----------------------------------------
---------------------------------
# Pixie — [ASSUMED] implemented as a plain
function per the dev spec's
# "implement or delete" open question. No
confirmed shape exists.
# ----------------------------------------
---------------------------------

def pixie_structure_goal(raw_request: str,
source_text: Optional[str] = None) -> dict:
    """[ASSUMED] Converts a human request
into a structured goal.
```

```
    Real Pixie logic is unconfirmed — this
is a passthrough stand-in."""
    return {"objective": raw_request,
"source_text": source_text}


# ----------------------------------------
--------------------------------
# EVACore — top-level orchestrator,
imported in api.py as `from eva.core import
EVACore`.
# ----------------------------------------
--------------------------------

class EVACore:
    def __init__(self, db_path: str =
"eva.db"):
        """[CONFIRMED shape:
EVACore(db_path)]"""
        self.db = Database(db_path)
        self.sentinel =
Sentinel(self.db)      # [CONFIRMED
attribute name]
        self.ghost =
Ghost(self.db)            # [CONFIRMED
attribute name]
        self.tools = ToolGateway(self.db,
self.sentinel)  # [CONFIRMED attribute
name]
        self.audit =
Audit(self.db)            # [CONFIRMED
attribute name]

    # --- Users ---------------------------
```

```
-----------------------------

    def create_user(self, name: str =
"anonymous") -> dict:
        user_id = new_id()
        self.db.conn.execute(
            "INSERT INTO users (id, name,
created_at) VALUES (?,?,?)",
            (user_id, name, now_iso()),
        )
        self.db.conn.commit()
        return {"id": user_id, "name":
name}

    # --- Missions ------------------------
-------------------------------

    def submit_mission(self, user_id: str,
objective: str, source_text: Optional[str]
= None,
                       sources:
Optional[list] = None, user_confirmed: bool
= False) -> dict:
        """[CONFIRMED via /mission/create
request/response shape]"""
        goal =
pixie_structure_goal(objective,
source_text)  # [ASSUMED — Pixie step from
spec §1]

        mission_id = new_id()
        self.db.conn.execute(
            "INSERT INTO missions (id,
user_id, objective, status, source_text,
```

```
user_confirmed, report, created_at) "
            "VALUES (?,?,?,?,?,?,?,?)",
            (mission_id, user_id,
goal["objective"],
MissionStatus.PENDING.value,
             source_text,
int(user_confirmed), None, now_iso()),
        )
        for s in (sources or []):
            self.db.conn.execute(
                "INSERT INTO
mission_sources (id, mission_id, origin,
description, text) VALUES (?,?,?,?,?)",
                (new_id(), mission_id,
s.get("origin"), s.get("description"),
s.get("text")),
            )
        self.db.conn.commit()

        return {
            "id": mission_id,
            "status":
MissionStatus.PENDING.value,
            "objective": goal["objective"],
            "report": None,
        }

    def get_mission_status(self,
mission_id: str) -> Optional[dict]:
        """[CONFIRMED to exist; exact
return shape was ❓ — ASSUMED here]"""
        row = self.db.conn.execute(
            "SELECT id, status, objective,
report FROM missions WHERE id = ?",
```

```
            (mission_id,),
        ).fetchone()
        return dict(row) if row else None

    # --- Tasks ---------------------------
---------------------------------

    def create_task(self, mission_id: str,
agent: str, description: str,
                    auth_level: str =
AuthLevel.USER_APPROVED.value) -> dict:
        task_id = new_id()
        self.db.conn.execute(
            "INSERT INTO tasks (id,
mission_id, agent, description, auth_level,
status, created_at) "
            "VALUES (?,?,?,?,?,?,?)",
            (task_id, mission_id, agent,
description, auth_level,
TaskStatus.PENDING.value, now_iso()),
        )
        self.db.conn.commit()
        return {"id": task_id, "status":
TaskStatus.PENDING.value}
```